

HSRU: Mathematical Verification of the Neuromorphic Split-Brain Spiking Recurrence Architecture

Sameer Humagain
Independent AI Researcher

October 5, 2026

Abstract

This paper rigorously derives and verifies the mathematical foundations of the Hybrid State Recurrent Unit (HSRU), a neuromorphic sequence model designed to achieve strictly constant $\mathcal{O}(1)$ inference memory while maintaining competitive language modelling capacity.

We establish three principal contributions. (i) We formally derive the Split-Brain architecture, proving that a biologically motivated decomposition into a spiking somatic channel and a continuous dendritic channel, combined with multiplicative gating, enables complete decoupling of digital latch logic from floating-point semantic compression. (ii) We prove that the causal Matrix Exponential Moving Average (EMA) in the Memory Layer provides a strictly bounded $\mathcal{O}(N^2D)$ state representation regardless of temporal horizon T , and derive exact closed-form gradients without materialising the $\mathcal{O}(TN^2D)$ trajectory tensor. (iii) We formalise the Spiking Linear Attention layer with Product Key Memory (PKM), demonstrating that Top- K sparse addressing on an orthogonally initialised Cartesian codebook provides a deterministic $\mathcal{O}(1)$ associative recall mechanism that resolves the Pigeonhole Principle within the fixed state manifold.

Contents

1	Introduction and Motivation	1
2	Architectural Primitives	1
2.1	Hardware-Compatible Activation Functions	1
2.2	The Neuromorphic Logic Layer	1
3	The Split-Brain Memory Layer	2
3.1	Channel Decomposition	2
3.2	Fused Input Projection	2
3.3	Sinusoidal Positional Clock	2
3.4	Dendritic Computation	2
3.5	Somatic Membrane Potential and Spiking	3
3.6	Output Projection	4
4	HSRUBlock: The Full Additive Forward Pass	4

5	The Spiking Linear Attention Layer with Product Key Memory	4
5.1	Motivation: Associative Recall and the Pigeonhole Principle	4
5.2	Orthogonally Initialised Codebook	5
5.3	Separable Product-Key Addressing	5
5.4	Chunked Causal State Update	5
6	Information Theory and the Data Processing Inequality	6
7	CUDA-Fused Gradient Derivation	6
8	Summary and Verification	6
9	Empirical Verification: HSRU Small on TinyStories	7
9.1	Training Configuration	7
9.2	Measured Training Metrics	7
9.3	Independent Reproducibility Checklist	8

1 Introduction and Motivation

Standard Transformer language models maintain an explicit Key-Value (KV) cache during autoregressive generation. For a sequence of length T , this cache has $\mathcal{O}(T)$ memory footprint and incurs $\mathcal{O}(T^2)$ computational cost during training. For recurrent networks such as LSTMs, the unbounded hidden state growth is avoided, but these architectures suffer from catastrophic forgetting as the sequence length grows because a fixed-size vector cannot losslessly compress an arbitrarily long history.

The HSRU architecture addresses both shortcomings through a biologically inspired Split-Brain design: a fixed-dimensional state matrix $\mathbf{S} \in \mathbb{R}^{N \times N \times D}$ acts as a compressed manifold for continuous associations, while a sparse spiking mechanism handles discrete, high-salience events. The key insight is that biological cortical circuits simultaneously operate in two regimes: dense, real-valued dendritic integration for semantic context, and sparse, all-or-nothing somatic spikes for discrete event encoding. HSRU instantiates both regimes in a single differentiable computation graph.

2 Architectural Primitives

2.1 Hardware-Compatible Activation Functions

The HSRU codebase defines hardware-accurate approximations of standard non-linearities to ensure bitwise consistency between the PyTorch training environment and the fixed-point C++ Edge Engine.

Definition 2.1 (Hardware GELU). The standard GELU activation is approximated by the piecewise quadratic:

$$\text{GELU}_{\text{hw}}(x) = \begin{cases} 0 & x \leq -2 \\ \frac{x}{2} + \frac{x^2}{8} & -2 < x < 2 \\ x & x \geq 2 \end{cases} \quad (1)$$

This approximation deviates from the true $\text{GELU}(x) = x\Phi(x)$ by at most 6×10^{-3} in absolute error over $x \in [-3, 3]$, which is negligible under 8-bit fixed-point quantisation.

Definition 2.2 (Hardware Sigmoid). The sigmoid is approximated by a piecewise linear clamp:

$$\sigma_{\text{hw}}(x) = \text{clamp}\left(0.5 + \frac{x}{8}, 0, 1\right) \quad (2)$$

This function is exactly representable in 8-bit arithmetic and eliminates all exponential computations from the inference path.

2.2 The Neuromorphic Logic Layer

Definition 2.3 (Gated Logic Unit (GLU)). The Logic Layer $\mathcal{L} : \mathbb{R}^D \rightarrow \mathbb{R}^D$ with intermediate width M computes:

$$\mathbf{g} = \sigma(\mathbf{W}_{\text{gate}} \mathbf{x}) \in (0, 1)^M \quad (3)$$

$$\mathbf{u} = \mathbf{W}_{\text{val}} \mathbf{x} \in \mathbb{R}^M \quad (4)$$

$$\mathcal{L}(\mathbf{x}) = \mathbf{W}_{\text{out}}(\mathbf{g} \odot \mathbf{u}) \in \mathbb{R}^D \quad (5)$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{D \times M}$ is initialised with $\mathcal{N}(0, 0.01^2)$ and its bias to $\mathbf{0}$, ensuring that the logic output is initially negligible and gradient flow begins through the identity path.

The fused implementation computes $[\mathbf{g}; \mathbf{u}] = F_{\text{linear}}(\mathbf{x}, [\mathbf{W}_{\text{gate}}; \mathbf{W}_{\text{val}}])$ in a single matrix multiplication, halving the number of BLAS kernel launches.

Remark 2.1. This is strictly a Gated Linear Unit, not SwiGLU. SwiGLU computes $\mathbf{g} \odot \text{SiLU}(\mathbf{u})$ where both projections receive independent linear transformations. The HSRU Logic Layer applies the gate as a pure sigmoid selector on the value stream.

3 The Split-Brain Memory Layer

3.1 Channel Decomposition

The Memory Layer $\mathcal{M} : \mathbb{R}^{B \times T \times D} \rightarrow \mathbb{R}^{B \times T \times D}$ maintains a hybrid internal state and partitions the hidden dimension D at index s :

Definition 3.1 (Split-Brain Partition).

$$\mathbf{h}_t = \underbrace{v_t^{(1:s)}}_{\text{Spiking Soma}} \parallel \underbrace{v_t^{(s+1:D)}}_{\text{Continuous Dendrite}} \quad (6)$$

Spiking channels maintain a perfect digital latch ($\alpha = 1$), functioning as bistable memory elements. Continuous channels apply a data-dependent leak $\alpha_t \in (0, 1)$, acting as exponentially decaying semantic accumulators. The split index $s = \lfloor D/2 \rfloor$ by default.

3.2 Fused Input Projection

Rather than computing five independent linear maps, the layer horizontally concatenates their weight matrices into a single fused projection:

$$\mathbf{F}_t = \mathbf{x}_t \cdot [\mathbf{W}_\alpha^\top \mid \mathbf{W}_\kappa^\top \mid \mathbf{W}_{in}^\top \mid \mathbf{W}_{d,in}^\top \mid \mathbf{W}_{d,\alpha}^\top] + \mathbf{b} \quad (7)$$

and then slices $\mathbf{F}_t$ into five components ($\hat{\alpha}_t, \hat{\kappa}_t, \hat{s}_t, \hat{d}_t, \hat{\delta}_t$), each $\in \mathbb{R}^D$. This reduces memory bandwidth overhead by a factor of 5 at the cost of a single, larger BLAS call.

3.3 Sinusoidal Positional Clock

To inject absolute positional information into the recurrent state without breaking the $\mathcal{O}(1)$ recurrent invariance, a sinusoidal positional clock is injected additively into the pre-activation input:

$$\mathbf{c}_{t,2i} = \sin\left(\frac{t}{10000^{2i/D}}\right) \quad (8)$$

$$\mathbf{c}_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/D}}\right) \quad (9)$$

The clock is cached with an LRU-1 policy: when the (seq_len, device, offset) key is identical across two consecutive calls—as it always is during TBPTT with fixed chunk size—the tensor is reused without recomputation, eliminating all trigonometric kernel launches.

3.4 Dendritic Computation

The dendritic state $d_{v,t} \in \mathbb{R}^D$ models a spatially integrated post-synaptic current that decays at an input-modulated rate:

$$d_{v,t} = \delta_t \odot d_{v,t-1} + \hat{d}_t \quad (10)$$

$$\mathbf{s}_t^{(d)} = \varphi\left(d_{v,t} - \vartheta^{(d)}\right) \quad (11)$$

$$d_{v,t} \leftarrow d_{v,t} - \mathbf{s}_t^{(d)} \odot \vartheta^{(d)} \quad (12)$$

where $\delta_t = \sigma(\hat{\delta}_t) \in (0, 1)^D$ is the learned dendritic decay, $\vartheta^{(d)} = \text{softplus}(\boldsymbol{\theta}_d) \in \mathbb{R}_{>0}^D$ is the learnable dendrite threshold with $\boldsymbol{\theta}_d$ initialised to 0.1, and $\varphi(\cdot)$ is the surrogate gradient fast sigmoid with slope 25:

$$\varphi(z) = \frac{1}{1 + e^{-25z}}, \quad \nabla_z \varphi(z) = \frac{25 \cdot e^{-25z}}{(1 + e^{-25z})^2} \quad (13)$$

The dendritic post-synaptic current (PSC) injected into the soma is:

$$\text{PSC}_t = \mathbf{s}_t^{(d)} \odot \mathbf{w}_{\text{dend}} \in \mathbb{R}^D \quad (14)$$

where $\mathbf{w}_{\text{dend}} \in \mathbb{R}^D$ is a learnable per-channel weight initialised to 0.1.

3.5 Somatic Membrane Potential and Spiking

The soma integrates the shunted input signal and the dendritic current into a membrane potential v_t . Critically, the shunting gate κ_t acts as a multiplicative inhibitor on the entire afferent signal (input plus dendrite), not on the previous membrane potential:

$$\mathbf{s}_{\text{eff},t} = (\hat{s}_t + \text{PSC}_t) \odot \kappa_t \quad (15)$$

$$v_t = \alpha_t \odot v_{t-1} + \mathbf{s}_{\text{eff},t} \quad (16)$$

where $\alpha_t = \sigma(\hat{\alpha}_t) \in (0, 1)^D$ is the leak gate and $\kappa_t = \sigma(\hat{\kappa}_t) \in (0, 1)^D$ is the shunting inhibition. The bias of $\mathbf{W}_\alpha$ is initialised to 1.0, yielding an initial leak $\alpha_0 \approx \sigma(1.0) = 0.731$. This is critical: with $\alpha_0 = 0.982$ (bias 4.0), the steady-state voltage variance is $\sigma_v^2 = \sigma_{in}^2 / (1 - \alpha_0^2) \approx 27\sigma_{in}^2$, causing universal saturation; with $\alpha_0 = 0.731$, we get $\sigma_v^2 \approx 1.13\sigma_{in}^2$, keeping the membrane below threshold for $\sim 90\%$ of inputs and achieving biologically realistic $\sim 10\%$ sparse firing.

Spiking Half ($1 \leq j \leq s$), $k = 1$ mode. In the default single-threshold configuration ($k_dims=1$), the spike is computed as a pure surrogate gradient approximation with no binary hard reset — the forward activation is the smooth sigmoid itself:

$$\mathbf{s}_t^{(j)} = \varphi(v_t^{(j)} - \vartheta^{(j)}) \in (0, 1) \quad (17)$$

The membrane reset subtracts the spike times the detached threshold:

$$v_t^{(j)} \leftarrow v_t^{(j)} - \mathbf{s}_t^{(j)}|_{\text{detached}} \odot \vartheta^{(j)} \quad (18)$$

Spiking Half ($1 \leq j \leq s$), thermometer $k > 1$ mode. When $k_dims > 1$, the model uses thermometer-encoded thresholds $\vartheta_1^{(j)} < \vartheta_2^{(j)} < \dots < \vartheta_k^{(j)}$ and the full Straight-Through Estimator (STE):

$$\mathbf{s}_t^{(j,k)} = \underbrace{\left[\left[v_t^{(j)} > \vartheta_k^{(j)} \right] \right]}_{\text{hard, forward}} - \underbrace{\varphi(v_t^{(j)} - \vartheta_k^{(j)})}_{\text{detached}} + \varphi(v_t^{(j)} - \vartheta_k^{(j)}) \quad (19)$$

This yields a true discrete $\{0, 1\}^k$ vector during the forward pass while routing gradients through the smooth surrogate. The membrane reset is proportional to the total number of threshold crossings.

The spike density is penalised by a mean-squared auxiliary loss against a biologically motivated target rate $\rho^* = 0.1$:

$$\mathcal{L}_{\text{aux}} = \left(\frac{1}{T \cdot B \cdot s} \sum_{t,b,j} s_{t,b}^{(j)} - \rho^* \right)^2 \quad (20)$$

Continuous Half ($s + 1 \leq j \leq D$). The continuous channels set the leak gate to 1 (perfect digital latch):

$$\alpha_t^{(j)} = 1.0, \quad s < j \leq D \quad (21)$$

These channels accumulate floating-point semantic associations without information-destroying discrete thresholding.

3.6 Output Projection

The final output of the memory layer reads both the somatic membrane trace $\mathbf{v}_t \in \mathbb{R}^D$ and the collapsed dendritic spike trace $\mathbf{d}_t \in \mathbb{R}^s$:

$$\mathbf{o}_t = \text{GELU}(\mathbf{w}_v \odot \mathbf{v}_t + \text{ZeroPad}(\mathbf{w}_d \odot \mathbf{d}_t) + \mathbf{b}) \quad (22)$$

followed by a LayerNorm, and then a residual connection back to the original input.

4 HSRUBlock: The Full Additive Forward Pass

Definition 4.1 (Additive Routing (Transformer-Parity)). Let $\mathbf{x}_t \in \mathbb{R}^{B \times D}$ be the input residual stream at layer ℓ . The HSRUBlock forward pass is:

$$\mathbf{m}_t, \mathbf{S}_t = \mathcal{M}(\text{LN}_1(\mathbf{x}_t), \mathbf{S}_{t-1}) \quad (23)$$

$$\mathbf{x}'_t = \mathbf{x}_t + \mathbf{\Sigma} \odot \mathbf{m}_t \quad (24)$$

$$\mathbf{l}_t, _ = \mathcal{L}(\text{LN}_2(\mathbf{x}'_t)) \quad (25)$$

$$\mathbf{x}_{t+1} = \mathbf{x}'_t + \mathbf{l}_t \quad (26)$$

where $\mathbf{\Sigma} \in \mathbb{R}^D$ is a learnable scaling parameter initialised to $0.1 \cdot \mathbf{1}$. The purpose of this near-zero initialisation is to establish a stable “double-silent” bootstrap: at step 0, both $\mathcal{M}$ and $\mathbf{\Sigma}$ output near-zero values, causing the model to begin training near an identity function, which prevents exploding activations in deep stacks.

Theorem 4.1 (Gradient Stability under Near-Zero Initialisation). In the limit $\mathbf{\Sigma} \rightarrow \mathbf{0}$, the Jacobian $\partial \mathbf{x}_{t+1} / \partial \mathbf{x}_t \rightarrow \mathbf{I}$, ensuring unit-magnitude singular values and preventing gradient vanishing or explosion at the start of training.

Proof. $\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{\Sigma} \odot \mathcal{M}(\cdot) + \mathcal{L}(\cdot)$. The Jacobian is $\mathbf{I} + \mathbf{\Sigma} \odot J_{\mathcal{M}} + J_{\mathcal{L}}$. Since $\mathbf{W}_{\text{out}} \sim \mathcal{N}(0, 0.01^2)$, we have $\|J_{\mathcal{L}}\|_F \approx 0$, and $\|\mathbf{\Sigma} \odot J_{\mathcal{M}}\|_F = 0.1 \|J_{\mathcal{M}}\|_F$. The dominant term is therefore $\mathbf{I}$, confirming the stability result. $\square$

5 The Spiking Linear Attention Layer with Product Key Memory

5.1 Motivation: Associative Recall and the Pigeonhole Principle

The somatic memory layer compresses all past information into a fixed-size manifold. By the Data Processing Inequality, this lossy compression prevents exact associative recall of arbitrary past tokens. The Spiking Linear Attention layer provides a complementary sparse addressing mechanism.

5.2 Orthogonally Initialised Codebook

The attention layer maintains a pair of orthonormal codebooks $\mathbf{C}_1, \mathbf{C}_2 \in \mathbb{R}^{H \times C \times (D/2)}$, where H is the number of heads and C is the number of codes per codebook. Each $\mathbf{C}_r[h] \in \mathbb{R}^{C \times (D/2)}$ is initialised via orthogonal initialisation, ensuring maximally spread initial code vectors:

$$\mathbf{C}_r[h]^\top \mathbf{C}_r[h] = \mathbf{I}_C \quad \forall r \in \{1, 2\}, h \in [H] \quad (27)$$

During the forward pass, codebooks are ℓ_2 -normalised to the unit sphere, maintaining hyperspherical uniform coverage of the query space as training proceeds.

5.3 Separable Product-Key Addressing

A query $\mathbf{q}_t \in \mathbb{R}^{D/2}$ (split into row-half $\mathbf{q}^{(1)}$ and column-half $\mathbf{q}^{(2)}$) computes inner-product scores against both codebooks:

$$r_{t,n} = \mathbf{q}_t^{(1)} \cdot \hat{\mathbf{c}}_{1,n}, \quad c_{t,n} = \mathbf{q}_t^{(2)} \cdot \hat{\mathbf{c}}_{2,n} \quad (28)$$

The Top- K row and column indices are retrieved:

$$(w_k^r, i_k^r)_{k=1}^K = \text{TopK}(\text{softmax}(\mathbf{r}_t)), \quad (w_k^c, i_k^c)_{k=1}^K = \text{TopK}(\text{softmax}(\mathbf{c}_t)) \quad (29)$$

This defines a K^2 Cartesian set of active cells in the $C \times C$ memory grid.

Theorem 5.1 ($\mathcal{O}(KC)$ Addressing Complexity). The PKM lookup has time complexity $\mathcal{O}(KC)$ and space complexity $\mathcal{O}(K^2)$ per query, independent of the sequence length T .

Proof. Computing $\mathbf{r}_t$ and $\mathbf{c}_t$ requires $2C \cdot (D/2) = CD$ multiplications. The TopK operation is $\mathcal{O}(C)$ with a min-heap. The outer product over K^2 cells requires $\mathcal{O}(K^2)$ space and $\mathcal{O}(K^2 D_h)$ multiplications for value accumulation, where $D_h = D/H$ is the head dimension. Since $K \ll C$ by design ($K = 2$, $C = 16$ in the small configuration), this is $\mathcal{O}(C)$ in the dominant term. $\square$

5.4 Chunked Causal State Update

During training in chunkwise-parallel mode, the KV state grid $\mathbf{G} \in \mathbb{R}^{H \times C \times C \times D_h}$ is written with a causal decay:

$$\Lambda_t = \exp\left(\sum_{\tau=1}^t \log g_\tau\right), \quad g_\tau = \sigma(\mathbf{W}_g \mathbf{x}_\tau) \in (0, 1) \quad (30)$$

$$\text{mask}_{st} = \exp(\Lambda_t - \Lambda_s) \cdot \mathbf{1}[s \leq t] \quad (31)$$

$$A_{st} = (\mathbf{R}_{Q,t} \cdot \mathbf{R}_{K,s})(\mathbf{C}_{Q,t} \cdot \mathbf{C}_{K,s}) \cdot \text{mask}_{st} / \sqrt{D_h} \quad (32)$$

The within-chunk output is:

$$\mathbf{o}_t^{\text{seq}} = \sum_{s \leq t} A_{st} \cdot \mathbf{v}_s \quad (33)$$

The cross-chunk (state) output is:

$$\mathbf{o}_t^{\text{state}} = \Lambda_t \cdot \text{AttnStateRead}(\mathbf{R}_{Q,t}, \mathbf{C}_{Q,t}, \mathbf{G}) \quad (34)$$

The state is updated at the end of each chunk:

$$\mathbf{G} \leftarrow \mathbf{G} \cdot g_T + \text{AttnKVUpdate}(\mathbf{R}_K, \mathbf{C}_K, \mathbf{v} \cdot \Lambda^{-1}) \quad (35)$$

6 Information Theory and the Data Processing Inequality

Theorem 6.1 (Upper Bound on Mutual Information). Let $\mathbf{X}_{1:T}$ denote the input sequence random variable and $\mathbf{S}_T$ the recurrent state at time T . Then:

$$I(\mathbf{X}_{1:T}; \mathbf{S}_T) \leq \text{Capacity}(N^2 D) = N^2 D \cdot \log_2 \mathcal{Q} \quad (36)$$

where $\mathcal{Q}$ is the numerical precision of the state (32 bits for single precision).

Proof. This follows directly from the Data Processing Inequality (DPI): for any Markov chain $\mathbf{X}_{1:T} \rightarrow \mathcal{M} \rightarrow \mathbf{S}_T$, we have $I(\mathbf{X}_{1:T}; \mathbf{S}_T) \leq H(\mathbf{S}_T)$, and $H(\mathbf{S}_T)$ is bounded by the support cardinality of the state space. $\square$

Definition 6.1 (Information Bottleneck Lagrangian). The optimal training objective for HSRU is the Information Bottleneck Lagrangian:

$$\mathcal{J}_{\text{IB}} = \max_{\theta} [I(\mathbf{S}_t; \mathbf{x}_{t+1}) - \beta I(\mathbf{S}_t; \mathbf{X}_{1:t})] \quad (37)$$

The leak gate $\mathbf{g}_t$ implicitly minimises $I(\mathbf{S}_t; \mathbf{X}_{1:t})$ by discarding low-relevance tokens, while the prediction head maximises $I(\mathbf{S}_t; \mathbf{x}_{t+1})$. The trade-off parameter β is encoded implicitly in the entropy regularisation via the cross-entropy language modelling loss.

7 CUDA-Fused Gradient Derivation

Theorem 7.1 ($\mathcal{O}(T)$ Exact Gradient Backpropagation). The exact parameter gradients of the Memory Layer can be computed in $\mathcal{O}(T \cdot N^2 \cdot D)$ time without materialising the temporal state trajectory $\{\mathbf{S}_0, \dots, \mathbf{S}_T\}$ in GPU VRAM.

Proof. Let $\mathbf{m}_t = \sum_{r,c} R_{Q,r} C_{Q,c} S_{r,c,d}$ be the state readout. The gradient with respect to the KV state is:

$$\nabla_{\mathbf{S}} \mathcal{L} = \sum_{t=1}^T (\nabla_{\mathbf{m}_t} \mathcal{L}) \otimes \mathbf{R}_{Q,t} \otimes \mathbf{C}_{Q,t} \quad (38)$$

This is a sum of T rank-1 outer products. The custom `AttnStateReadFn` autograd function saves only the raw inputs $(\mathbf{R}_Q, \mathbf{C}_Q, \mathbf{G})$ during the forward pass—not the intermediate $\mathbf{Q}_{\text{outer}} = \mathbf{R}_Q \otimes \mathbf{C}_Q$, which would cost $\mathcal{O}(TN^2)$ VRAM. During backward, the two-step einsum contraction is recomputed inside L1 SRAM:

$$\text{step1}_{b,t,r,d} = \sum_c C_{Q,b,t,c} \cdot S_{b,r,c,d} \quad (39)$$

$$m_{b,t,d} = \sum_r R_{Q,b,t,r} \cdot \text{step1}_{b,t,r,d} \quad (40)$$

This gradient checkpointing strategy recomputes `step1` from saved inputs, exchanging $2 \times$ FLOPs for $\mathcal{O}(N)$ rather than $\mathcal{O}(N^2)$ activation storage, achieving standard Transformer spatial complexity during training. $\square$

8 Summary and Verification

The mathematical derivations in this document are verified to correspond exactly to the implementation in `dynamic_hsrupy`. All equations are derived directly from the PyTorch forward and backward passes of `HSRUMemoryLayer`, `HSRULogicLayer`, `HSRUSpikingAttentionLayer`, and the CUDA autograd functions `AttnStateReadFn` / `AttnKVUpdateFn`.

Table 1: Complexity comparison: Transformer vs. HSRU Small (26.3M parameters)

Property	Transformer	HSRU
Inference Memory	$\mathcal{O}(T)$ KV-Cache	$\mathcal{O}(1)$ State Matrix
Training Compute	$\mathcal{O}(T^2D)$	$\mathcal{O}(TDN^2)$
Gradient Storage	$\mathcal{O}(TN^2D)$	$\mathcal{O}(TD)$ (checkpointed)
Spike Density	N/A	$\sim 10\%$ (sparse)
Associative Recall	Full Attention	Top- K PKM
Edge Deployable	No (growing cache)	Yes ($\mathcal{O}(1)$ C++ binary)

9 Empirical Verification: HSRU Small on TinyStories

To validate that the mathematical claims in this document correspond to a genuinely trained model — not a theoretical construction — we record the complete empirical training results for the HSRU Small configuration (26.3M parameters) on the TinyStories dataset mixture.

9.1 Training Configuration

Table 2: HSRU Small hyperparameter configuration

Hyperparameter	Value
Architecture	12 HSRU Blocks, $D = 256$, $M = 1024$
Vocabulary	50,257 tokens (GPT-2 BPE)
Context (TBPTT chunk)	1024 tokens (256 chunk size)
Dataset	TinyStories pretrain mixture
Total Training Steps	16,000
Batch Size	32
Peak Learning Rate	1.06×10^{-3} (Cosine decay)
Min Learning Rate	1.06×10^{-4}
Warmup Ratio	2% (320 steps)
Weight Decay	0.1
Gradient Clip	1.0
Mixed Precision	bfloat16

9.2 Measured Training Metrics

All values below are taken directly from the raw training telemetry log (Metrics/LLM_Log.csv) which accompanies this release.

Table 3: Measured training metrics at key checkpoints

Step	Class Loss	Perplexity	Spike Density	Steps/sec
1 (init)	10.958	57,397	2.3%	0.05
100	~ 5.2	~ 181	$\sim 8\%$	1.17
8,000	~ 1.5	~ 4.5	$\sim 10\%$	1.17
16,000 (final)	1.378	3.966	10.48%	1.17
Validation (step 16,000)	1.232	3.427	—	—

Key Observations.

1. Convergence: The class loss descends from 10.958 (random initialization, $\log_2 50257 \approx 15.6$ bits) to 1.378 over 16,000 steps, demonstrating successful learning of the TinyStories distribution.
2. Spike Density: The measured spike density stabilises at $\approx 10.48\%$ after warmup, confirming that the auxiliary loss $\mathcal{L}_{\text{aux}}$ correctly drives the network toward the target biological firing rate $\rho^* = 0.10$.
3. Validation Generalization: The validation perplexity of 3.427 is lower than the training perplexity of 3.966, confirming the model is not overfitting and generalizes to unseen TinyStories documents.
4. $\mathcal{O}(1)$ Inference Footprint: During generation, the model maintains a constant GPU state of $\approx 1,598$ MB regardless of sequence length, confirming the memory-bounded recurrent architecture.

9.3 Independent Reproducibility Checklist

Any researcher can independently verify all claims in this document using the pre-compiled binary distributions included in the Demos/ folder:

1. Verify $\mathcal{O}(1)$ Runtime: Run the macOS demo and observe that GPU/CPU RAM remains flat as generation length grows. To achieve the peak 126 tokens/sec benchmark, run the raw C++ executable:

```
cd Demos/Mac_HSRU_Small
./hsru_edge
```

Alternatively, you can query the engine via the Python API wrapper (python3 test_api.py).

2. Verify Live Language Generation: The demo will stream tokens from the compiled C++ Black Box engine. Token throughput is ≥ 126 tokens/sec on Apple Silicon.
3. Inspect Raw Training Telemetry: The full 16,000-step training log is included at Metrics/LLM_Log.csv. All columns (loss, perplexity, spike density, learning rate, gradient norm, RAM usage) are recorded at every step.
4. Inspect Training Dashboard: A visual Neural Dashboard summary of the full training run is available at Metrics/LLM_Dashboard.png.
5. Cross-reference Architecture: The mathematical equations in this document correspond exactly to HSRUMemoryLayer in dynamic_hsr.py. The PyTorch training weights are publicly available at huggingface.co/hsameer/hsru-small.